



Εθνικό και Καποδιστριακό Πανεπιστήμιο Αθηνών
Τμήμα Πληροφορικής και Τηλεπικοινωνιών

Παράλληλα Συστήματα
Εργασία Simd

Ονοματεπώνυμο: Ευάγγελος Τραϊκόγλου

Αριθμός Μητρώου: 1115202200189

- cpu: i7-8700
- cores: 12 logical, 6 physical
- os, kernel: Ubuntu 24.04.3 LTS, 6.8.0-94-generic
- comp version: gcc 13.3.0

Polynomial Multiplication

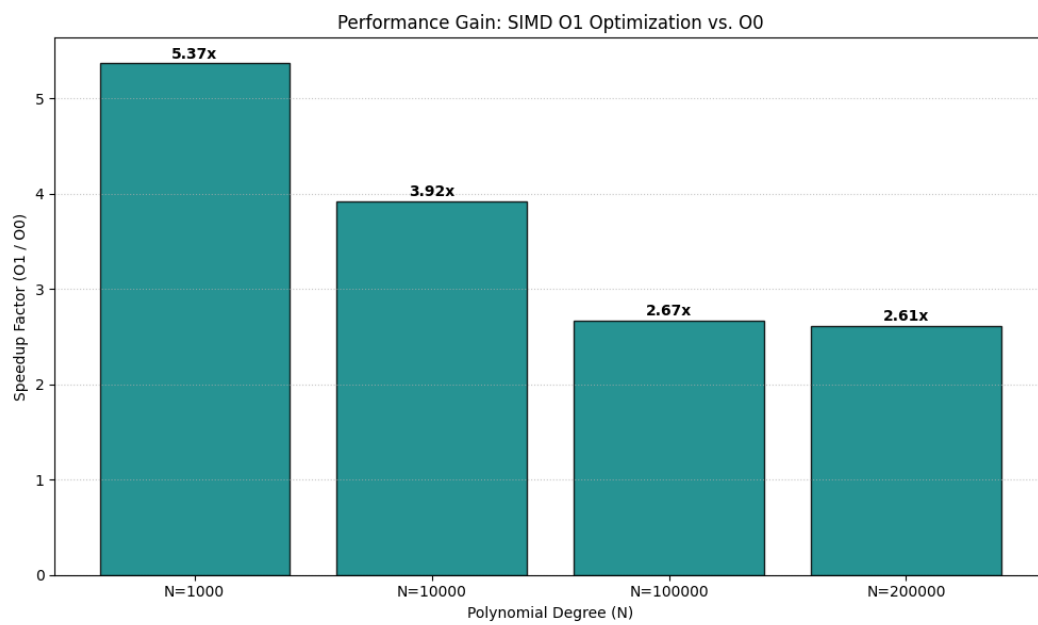
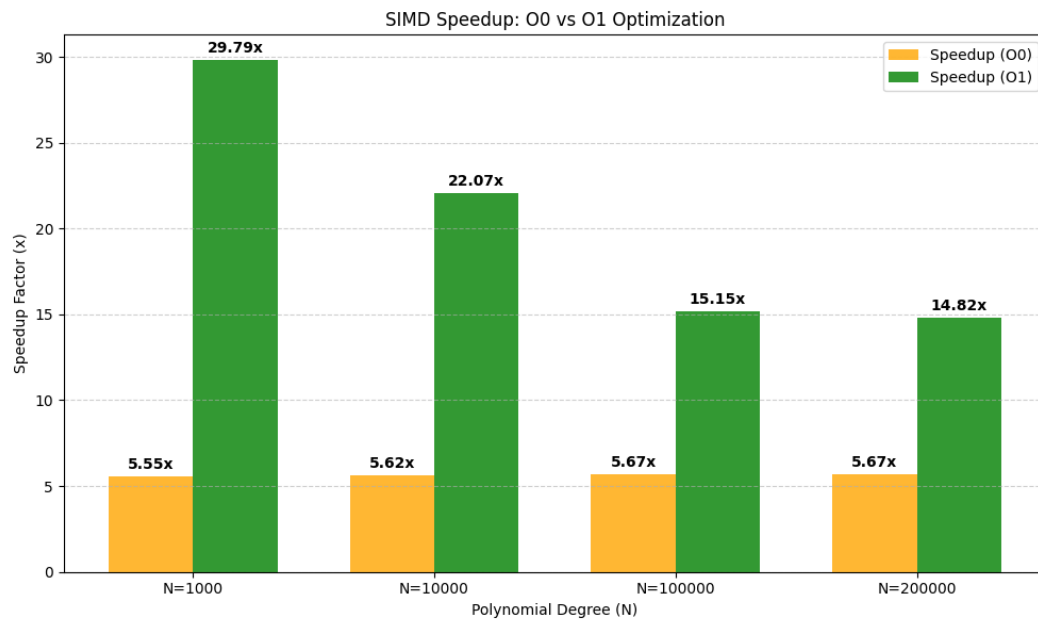
Η άσκηση επικεντρώνεται στον πολλαπλασιασμό πολυωνύμων βαθμού n . Ο σειριακός αλγόριθμος ακολουθεί την απλή υλοποίηση $O(n^2)$ χρόνου. Για την παράλληλη εκδοχή υπάρχουν 2 υλοποιήσεις γιατί παρατηρήθηκε ότι με τα optimize flags off κάθε update που γινόταν σε 1 register είχε ως αποτέλεσμα την άμεση καταγραφή και στο stack παρόλο που δεν χρειαζόταν αφού θα χρησιμοποιούνταν αμέσως μετά. Αυτό ήταν το κύριο bottleneck. Απ' ότι διάβασα είναι για λόγους debugging. Οπότε υπάρχει και 2 υλοποίηση (simd_mul_O1) που χρησιμοποιεί το O1 flag και η διαφορά στο χρόνο είναι σημαντική: $\sim 2.3\text{sec}$, 100k degree input. Στην simd_mul_O1, χρησιμοποιούνται simd εντολές 256bit/8 elements (load,store,add,multiply). Ακόμη ο βρόχος ξετυλίγεται (4 επαναλήψεις, saves 2.2sec on 100k degree input).

Σημείωση: Το base για τα optimizations που εξηγούνται παρακάτω είναι η simd_mul_O1 χωρίς το directive για optimize.

Στην simd_mul έγιναν κάποιες βελτιστοποιήσεις με το χέρι. Όπως ειπώθηκε το βασικό πρόβλημα είναι τα αχρείαστα store στο stack. Για αυτό το λόγο προστέθηκε το directive ACCUMULATE_8 που εμφωλευμένα καλεί όλα τα calls που πρέπει να γίνουν για την δάδα στοιχείων (load,mult,add,store). Αυτό εξαλείφει μερικά stack writes και ρίχνει τον χρόνο ($\sim 0.6\text{sec}$ για 100k degree pol). Ακόμα το pointer arithmetic για να βρεθεί η επόμενη θέση έναντι του απλού access, eg: $a[i+j]$ ρίχνει τον χρόνο κατά 1.3sec, 100k pol-deg. Τέλος, το prefetch του επόμενου cache line ρίχνει το χρόνο κατά: $\sim 0.2\text{sec}$ 100k, $\sim 1\text{sec}$ 200k.

Run: `./polynomial_mult -n <degree>`

Σημείωση: Όλα τα πειράματα υπάρχουν στο csv αρχείο: simd_bench_results.csv.



Συμπεράσματα: Η επιτάχυνση που παρατηρείται είναι σταθερή (~ 5.6) για όλα τα μεγέθη πολυωνύμων αφού χρησιμοποιείται το ίδιο μοτίβο γενικά της επεξεργασίας 8 στοιχείων τη φορά μέσω vector registers. Το βασικό bottleneck στη περίπτωση του `simd_mul` είναι το memory traffic λόγω των αχρείαστων stack writes που αν και μειώθηκε λόγω των βελτιστοποιήσεων, δεν εξαφανίστηκε πλήρως. Προφανώς στη περίπτωση O1 και άλλες βελτιστοποιήσεις μπορεί να βοηθήνε συμπληρωματικά (πχ καλύτερο scheduling εντολών) για αυτό και παρατηρείται μεγάλη διαφορά μεταξύ των 2 υλοποιήσεων (κυρίως στα μικρά μεγέθη), ενώ σταθεροποιείται περίπου στο ~ 2.5 η επιτάχυνση O1 έναντι του O0 στα μεγαλύτερα.